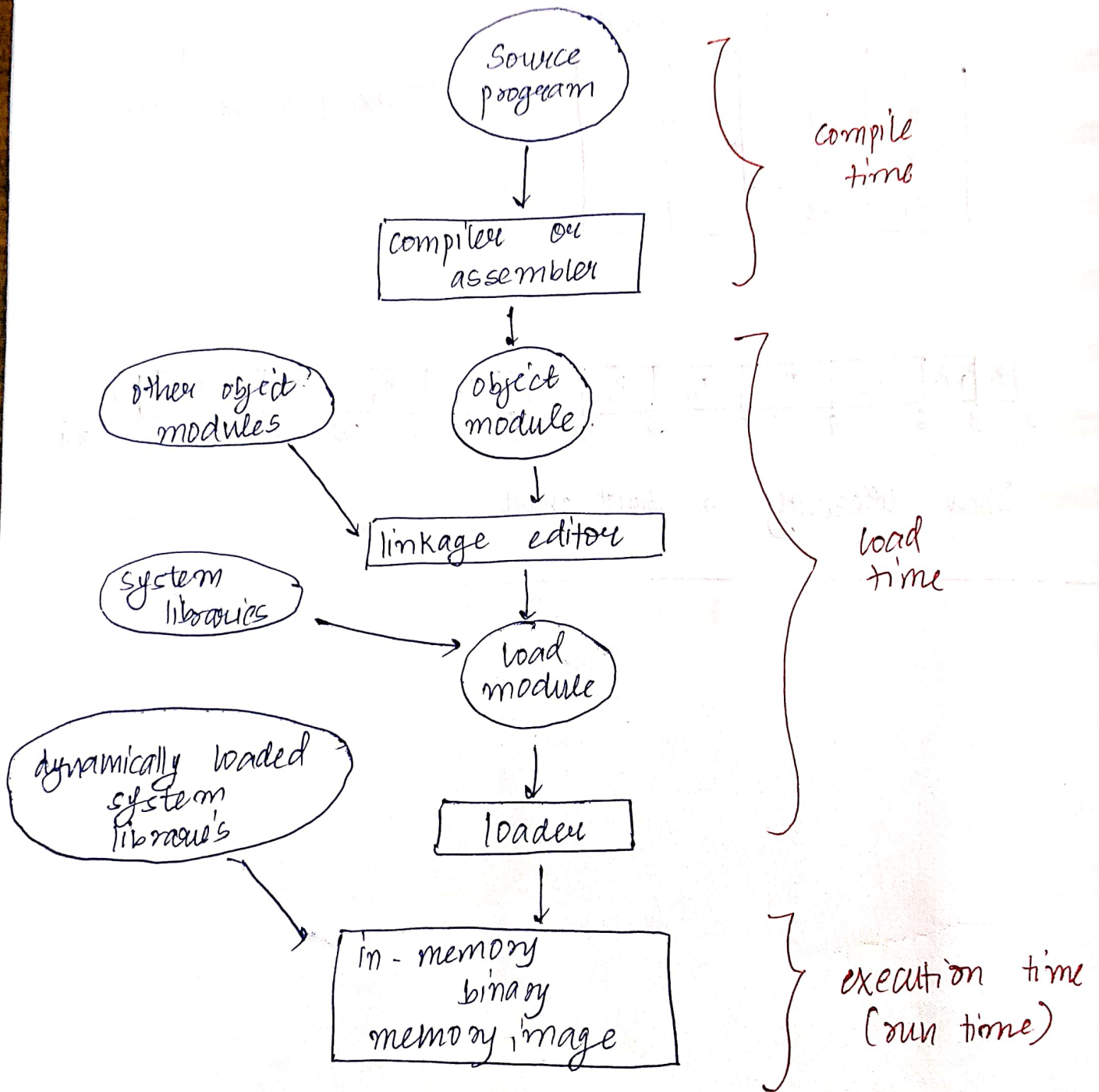


→ MEMORY

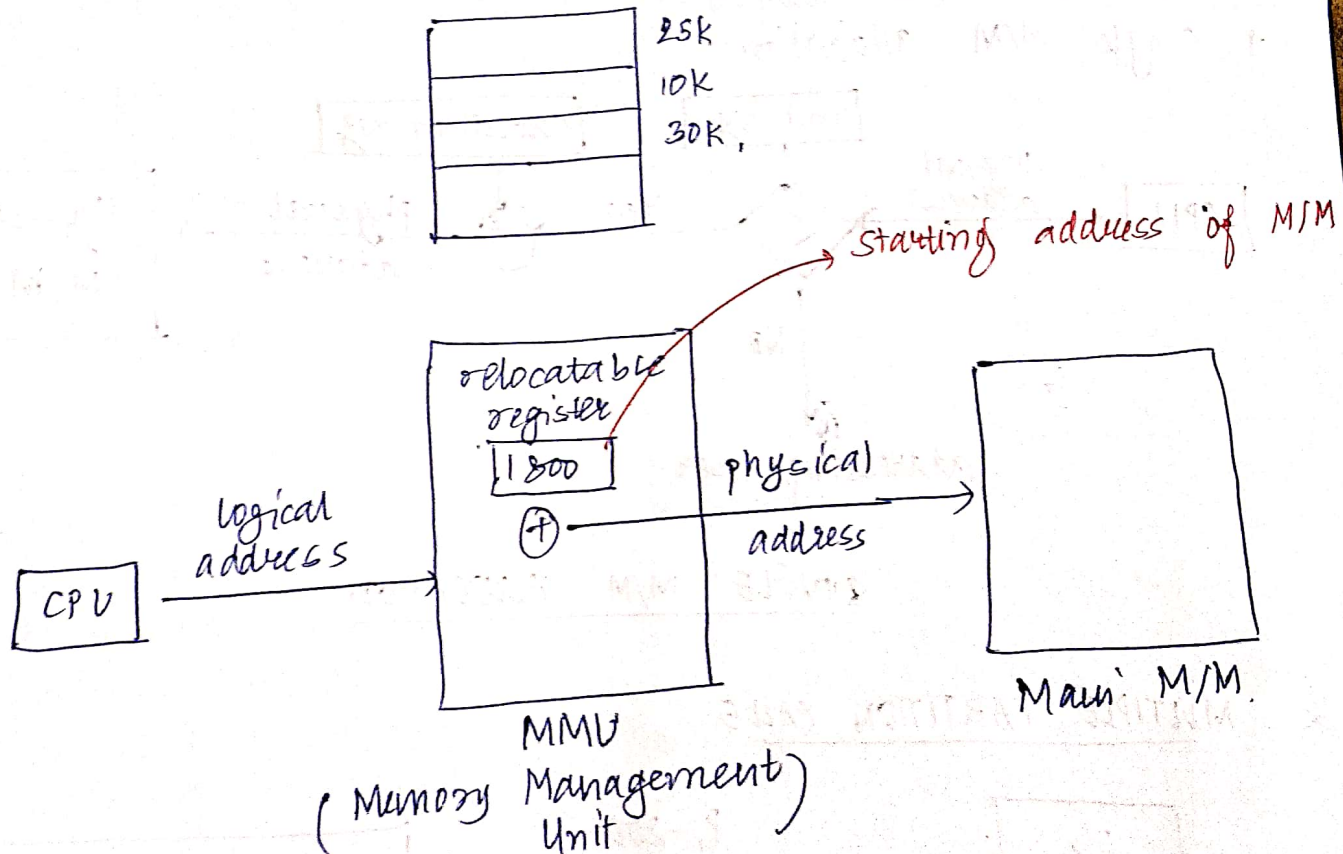
* Address binding : fixing addresses for executable files



* Dynamic loading : The routine is not directly loaded into the M/M. It is loaded from the addresses where it is present.

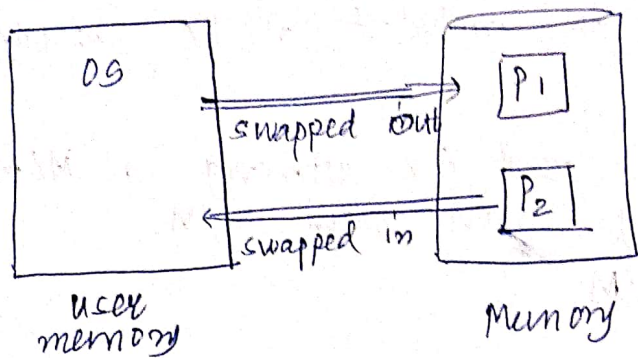
* Dynamic linking: The routine is not linked until it is called. A stub is included in the library routines.

* Overlays: Technique used for defining a M/M that can be used along with main M/M.
→ single piece of M/M
or
chunks of M/M

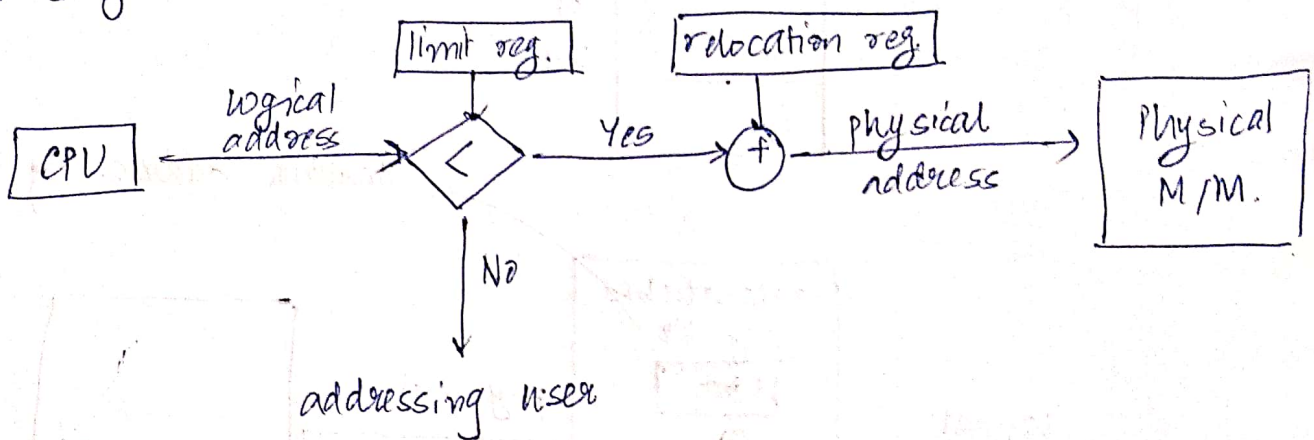


port / routine
used for conversion b/w
logical & physical add.

* Swapping: needs addresses & diff. scheduling algos.

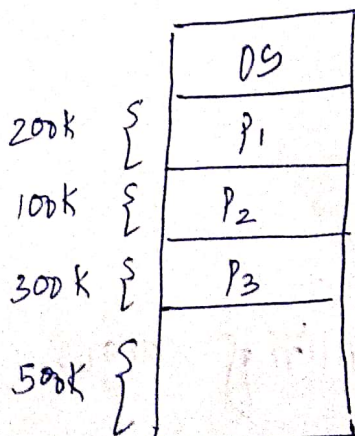


- * Continuous M/M allocation
- * Single M/M allocation



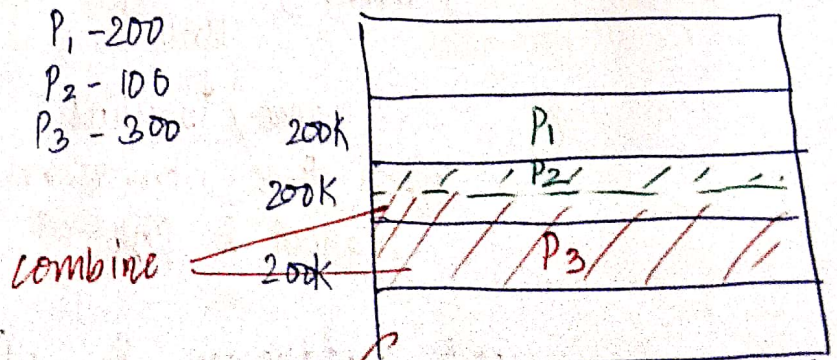
SINGLE M/M ALLOCATION

* MULTIPLE PARTITION CALLS



fixed size
(processes)

P₁ - 200
P₂ - 100
P₃ - 300



these chunks are known as
holes / unused spaces

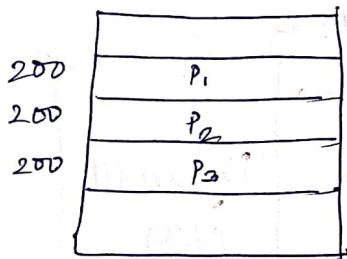
fixed size partitions

- * ALGORITHMS :-
- first fit
 - Best fit
 - Worst fit (Random)

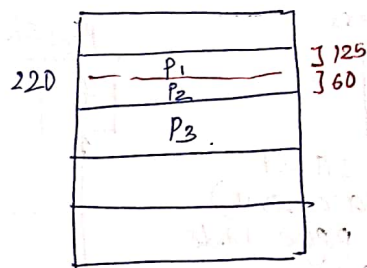
$P_1 - 125$

$P_2 - 60$

$P_3 - 185$



FIRST FIT



BEST FIT

* FRAGMENTATION

Internal

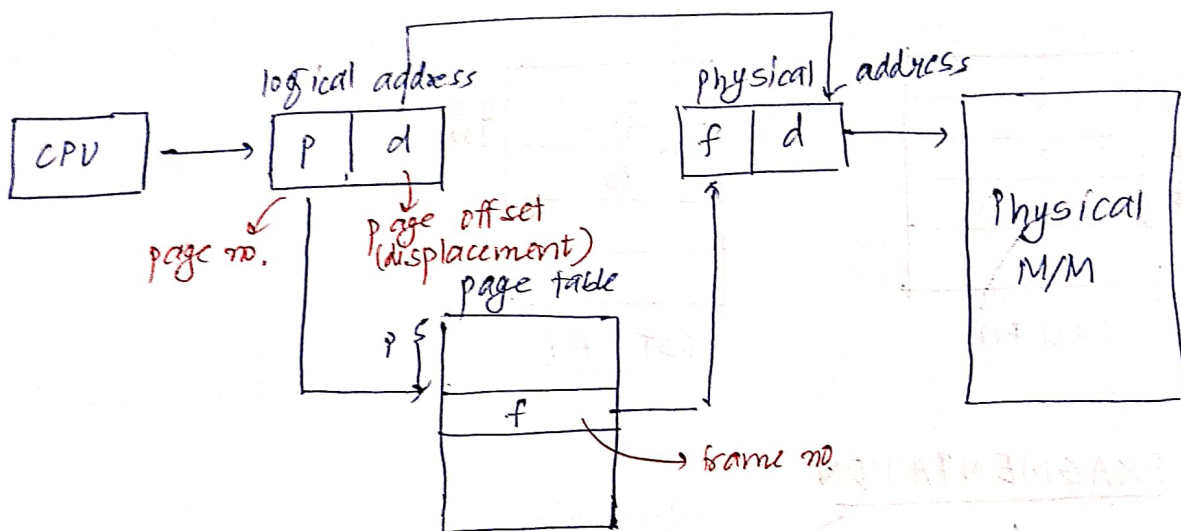
External

to prevent internal fragmentation, small processes are combined to use a chunk of M/M. This is known as COMPACTION.

→ PAGING

↓
dividing the logical M/M into blocks called pages

* Physical M/M is divided into frames.



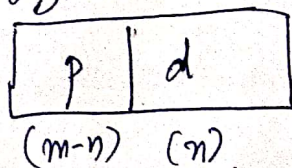
paging H/W

→ Hardware

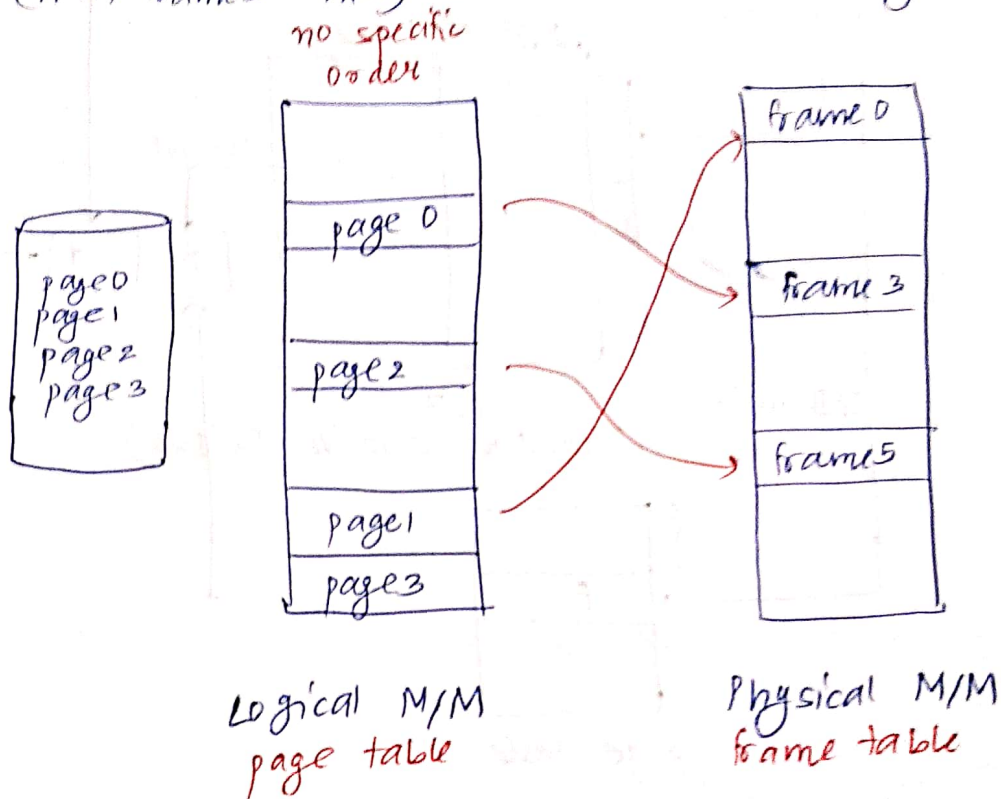
* size of page $\equiv$ power of 2
 $\equiv 512 - 8192$ bytes

If logical address is 2^m & page size is 2^n , then
higher (m-n) bits of logical address are used for page no.

logical address



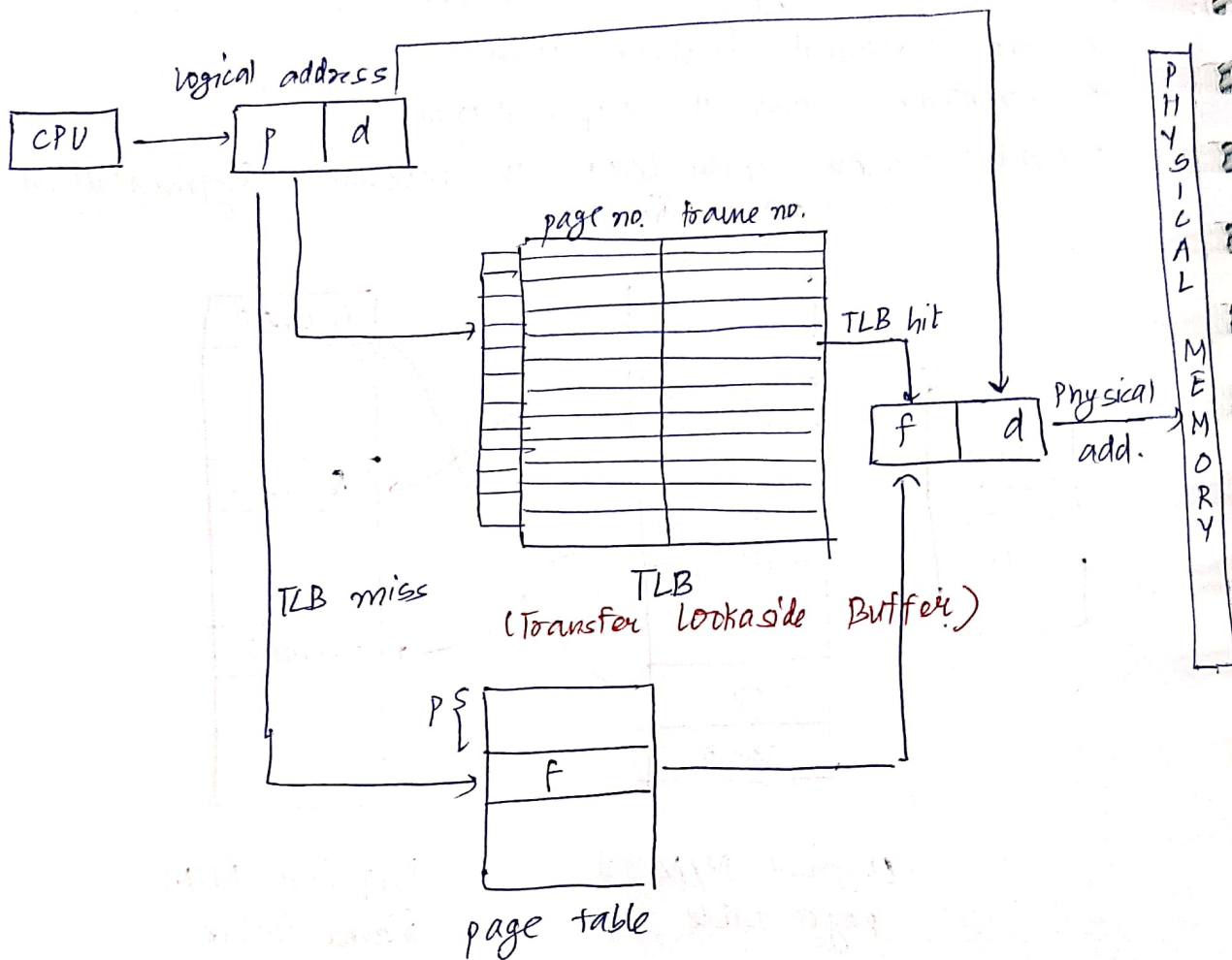
- * Paging is dynamic allocation
- * No External fragmentation
- * Sometimes, internal fragmentation
- * $(n+1)$ frames may lead to internal fragmentation.



ADDRESS TRANSLATION H/W

↳ to map logical add. & physical add.
gives page nos. & frame nos.

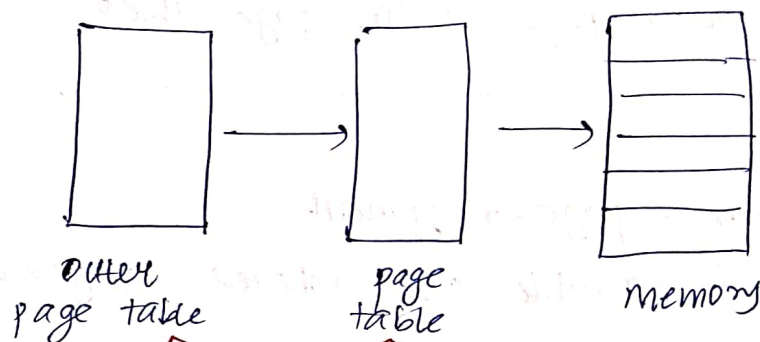
- * Page table is implemented with registers, which are high speed.
- * Page table is kept in main M/M along with Page Table Base Register (PTBR)
- * Other buffer register consists of key & value.
- * Translation Look Aside Buffer tells how to map physical address & logical address.



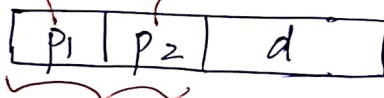
* In page table, along with frame no., we can have validity bit to check its existence.

frame no.	Validity bit

* In case of multiple paging, we have multiple page tables.



MULTILEVEL
PAGING

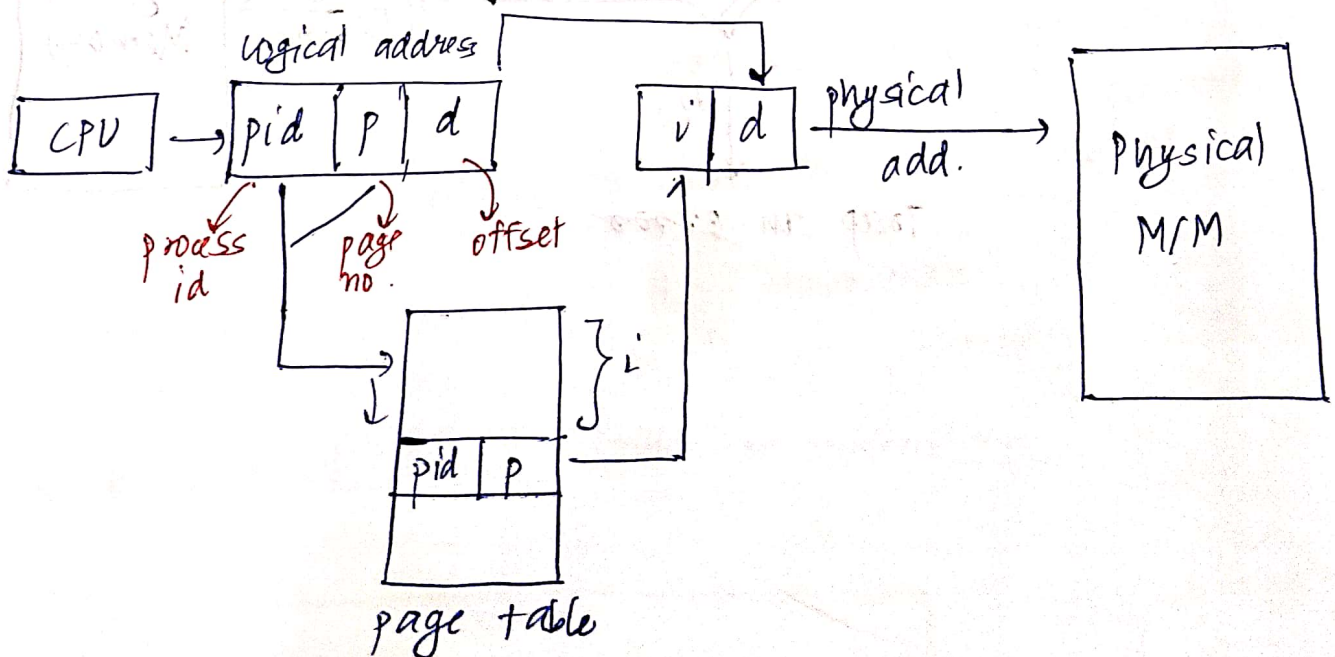


page no. is divided into portions

* INVERTED PAGE TABLE

If there are millions of places, large physical storage is used. Thus, we use inverted page table

- One entry for every real page
- Use inverted address



* SHARED PAGES

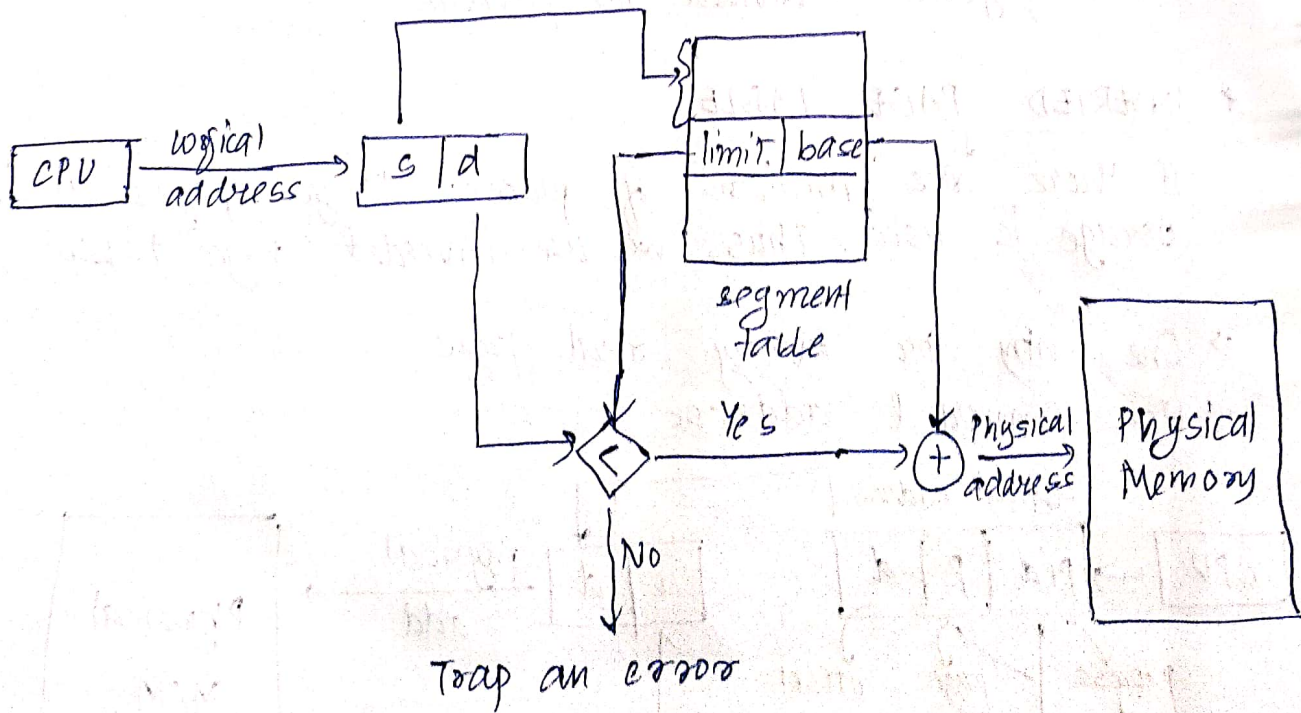
↓
multiple processes share pages
- same page no. for a page in diff. page tables

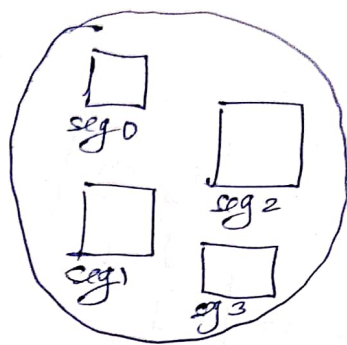
→ SEGMENTATION

sector → block → frame → page → segment

* Segments are of variable size whereas pages are of same size.

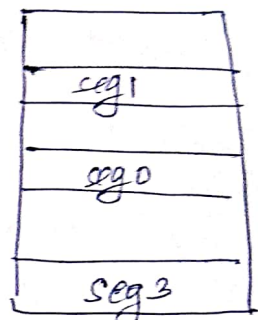
* segment_no, offset
(s) (d)





limit	base

segment table



Physical M/M

Starting address = 4300

53, seg 2

$$4300 + 53 = 4353$$

* STBR : Segment Table Base Register
 ↓
 points towards segment table (address)

* STLR : Segment Table Length Register
 ↳ gives the length of the segments

* SHARED SEGMENTS.

* Segments have external fragmentation

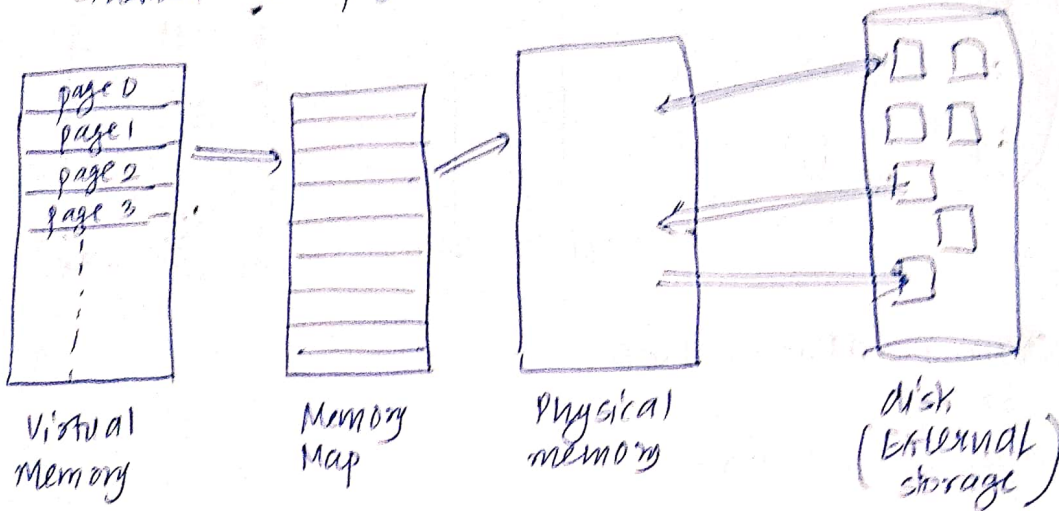
∴ they are large size of blocks
 some of the blocks remain unused
 segments cannot be combined

* To prevent external fragmentation:

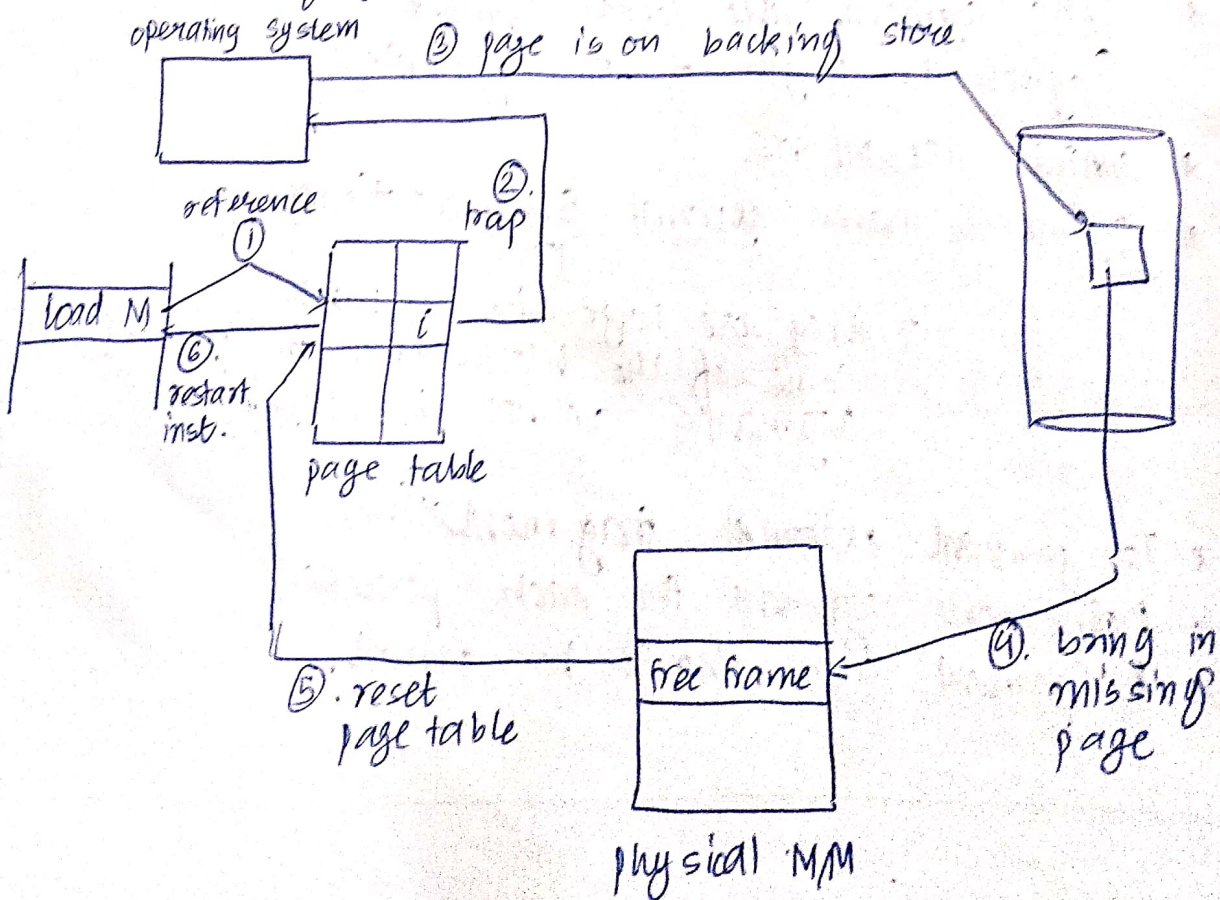
- (i). Define one segment for each process.
- (ii). Segment size should be small.

→ VIRTUAL MEMORY

Divided into pages



- * Swapping process is very important.
- * Valid/Invalid Bit
- * Demand Paging



Steps :

- ①. Check an internal table which is kept with PCB to determine whether the reference bit was a valid/invalid memory access.
- ②. If the reference bit was invalid, terminate the process.
If it was valid, but not in page table, then bring it.
- ③. We find a free frame from the frame list
- ④. Schedule a disk operation to read the desired space into the new allocated frame
- ⑤. When the disk read is complete, modify the internal table kept with the process & page table to indicate that the page is now in M/M.
- ⑥. Restart the instruction that was interrupted by illegal address trap.

Now the process can access the page.

Prob. of page fault

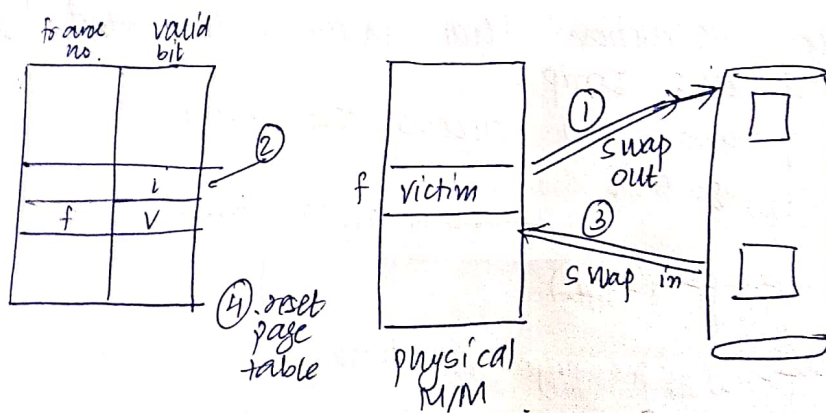
M/M access time

$$\text{Effective access time} = (1-p) \times m_a + (p \times \text{page fault time})$$

↪ no. of pages undergoing fault

→ PAGE FAULT ROUTINE

1. Find the location of the desired page on the disk.
2. Find the free frame.
 - (a). If there is a free frame, use it.
 - (b). Otherwise, use a page replacement algorithm to select a victim ~~page~~ frame.
 - (c). Write the victim page to the disk & change the frame & page tables accordingly.
3. Read the desired page into the newly free frame & change page & frame tables.
4. Restart the user process.



→ PAGE REPLACEMENT ALGOS

- ①. FIFO
- ②. Optimal
- ③. LRU
- ④. LRU approx.
- ⑤. Counting
 - LFU
 - MFU
- ⑥. Page buffering

→ PAGE REPLACEMENT ALGOS

1. FIFO

Sequence of page nos. :

D100, D432, D101, 0612, D102, D103,
 D404, D101, 0611, D102, D103, D104
 D101, 0610, D102, D103, D104, D101
 D609, D102, D105, 0510, D426, D232, D242

Reference string: (~~size of~~ ^{no.} page = 100 ~~bytes~~)

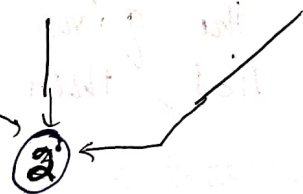
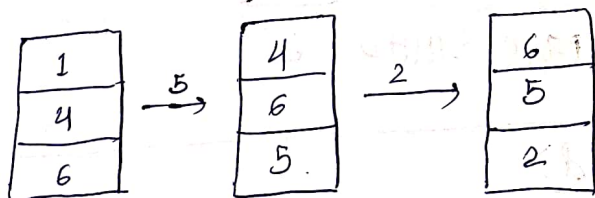
→ divide each page by 100 to get the Reference string

1, 4, 1, 6, 1, 1, 4, 1, 6, 1, 1, 1, 6, 1, 1, 1, 1, 6, 1, 1, 5, 4, 2, 2

Let us assume that total no. of pages that can be loaded at a time = 3

→ 3 pages correspond to 3 frames

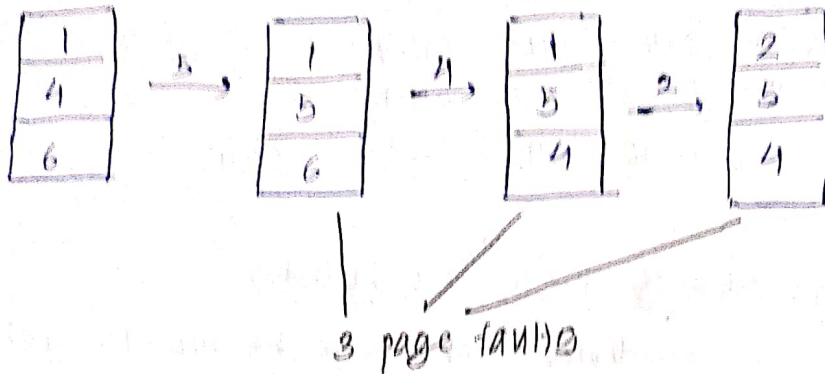
frames → 3



→ Total no. of page faults
 (Total no. of replacements)

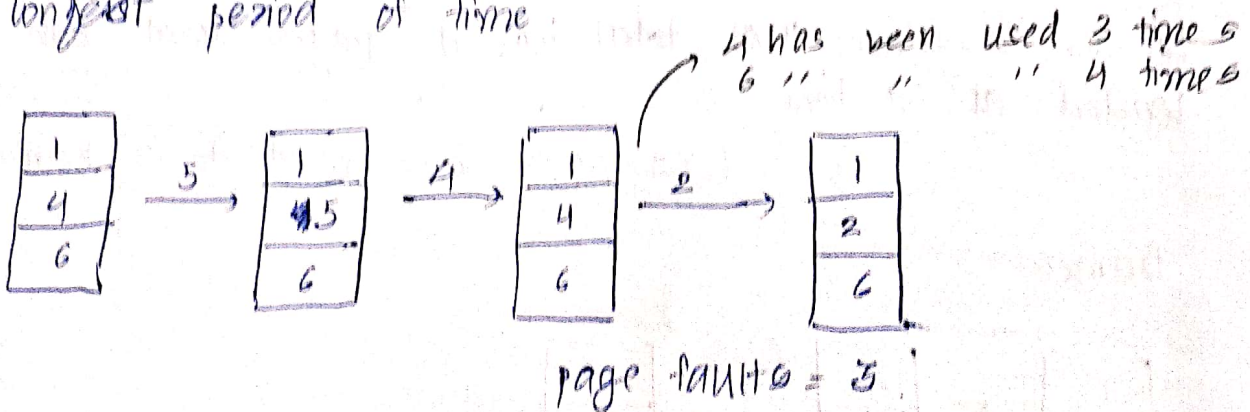
②. Optimal

If the page is not reqd. for a long period of time, then the page is replaced.



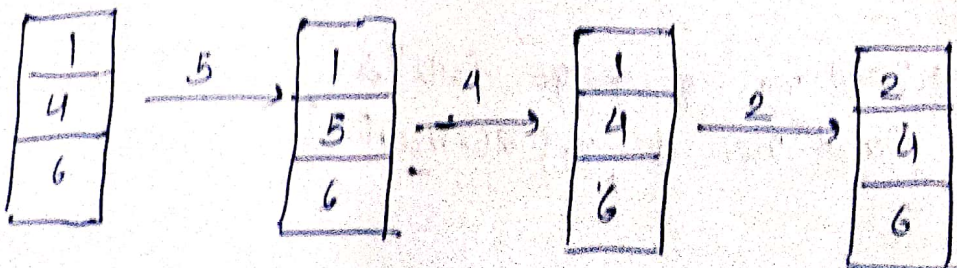
③. LRU (Least Recently Used)

Replace the page that has not been used for longest period of time



④. MFU (Most Frequently Used)

Replace the place just after the given page. If the page is not in the frame list, then apply FIFO.



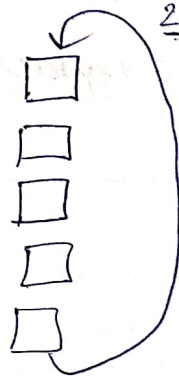
⑤. LRU approx.

* Reg. value = 10001001
Shift Right the reg. when page fault occurs.

10001001
01000100
00100010

* Also, we can use a single bit registers & then rotate the bits.

2nd chance LRU approx. algo.



(circular queue).

* Enhanced 2nd chance algo

2 bit information is used along the frame no.

(0,0) = Neither recently used nor modified

(0,1) = Not recently used, but modified.

(1,0) = Recently used, but clean

↳ can be used again

(1,1) = Recently used and modified

⑥. Counting algo

* LRU

count will be maintained that how many times the page has been referenced.
→ The page having the max. count will be replaced.

* MRU

reference count

→ The page having min. count will be replaced

⑦. Page Buffering Algo

Keep a list of free frames.

→ MULTIPROGRAMMING

Algo → equal allocations

93 frames → 5 processes

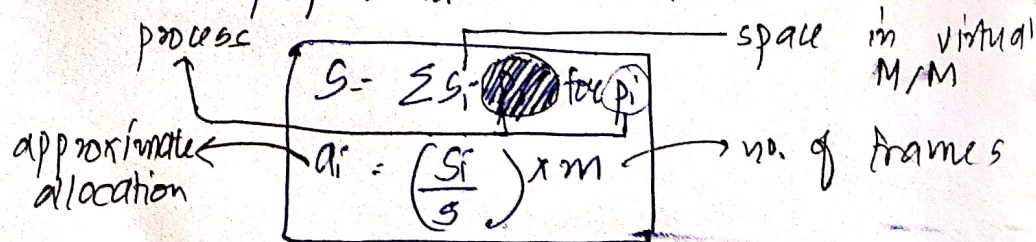
$$\frac{93}{5} = 18 \text{ frames for each process}$$

* Limitation / Anomaly : frames are allocated, but may not be used

(BALADAYS ANOMALY)

⇓

proportional allocation



eg: $S = \sum S_i$
 $= 127 + 10 = 137$ pages
 $m = 62$ frames
 1st process $= \frac{10}{137} \times 62$
 $= 4$ frames

2nd process $= \frac{127}{137} \times 62$
 $= 52$ frames

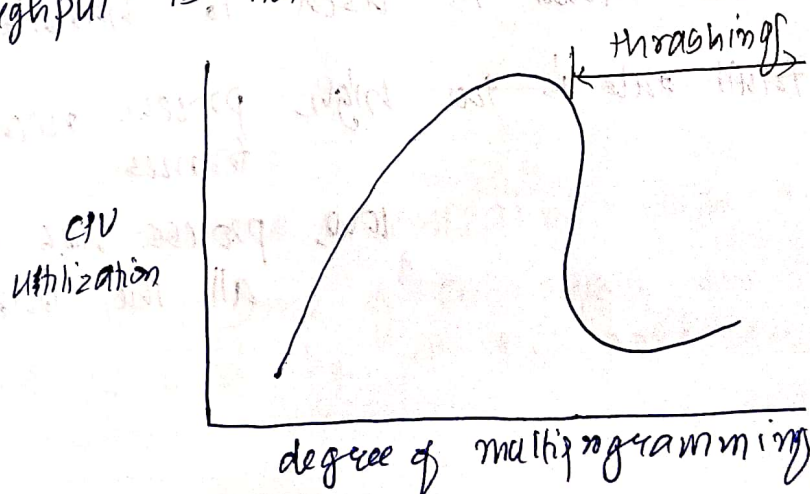
* Global & Local Allocation

→ No. of frames allocated cannot be changed
 cannot control page fault

* THRASHING (higher paging activity)

Replacement of pages is fast $\rightarrow$ page fault $\uparrow$
 $\downarrow$
 Thrashing

- > Global page table
- > CPU utilization $\downarrow$
- > Throughput is not to the maximum



• To prevent thrashing, we use a strategy working set Model

working set window (Δ)
 ↳ page references

Page reference list:

2. 6 1 5 7 7 7 7 7 5 5 6 2 3 4 1 2 3 4 4 4 4
 ← Δ → ← Δ →

$\Delta = 10$ memory

Working set size

$t_1 = \{1, 2, 5, 7\}$, $WS(t_1) = 5$

$t_2 = \{3, 4\}$, ~~$WS(t_2) = 2$~~ ($WS(t_2) = 2$)

↳ 1, 2, 4 is referenced before in t_1 already

• If $\Delta \rightarrow$ too large

↳ too small

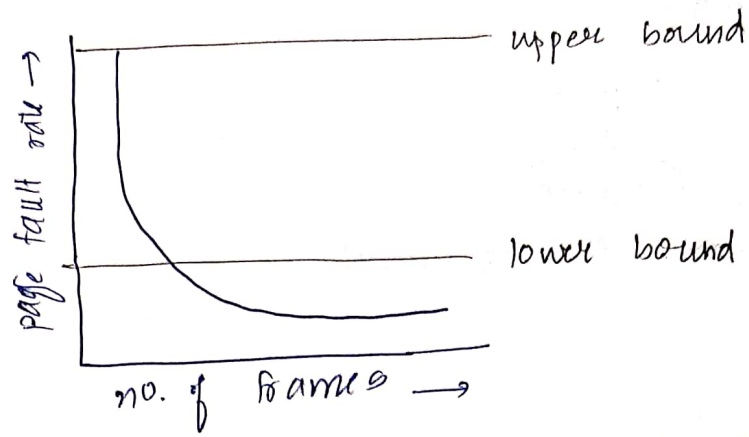
• For each process, a total demand for pages (D) is calculated

$$D = \sum (WS(t_i))$$

• Working set Model is used to handle thrashing

• If page fault rate is too high, process needs more frames

low, process has occupied all the frames



- If page fault ^{rate} exceeds upper bound, allocate another frame to the process.
- If page fault rate drops below lower limit, remove the frame from the process.

→ Other Things

* Prepaging : to attempt to prevent high level of initial paging

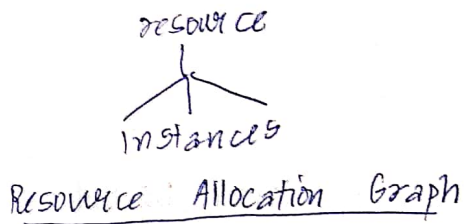
* ~~Page size~~

$$\boxed{\text{Total cost of prepaging} = (1 - \alpha)}$$

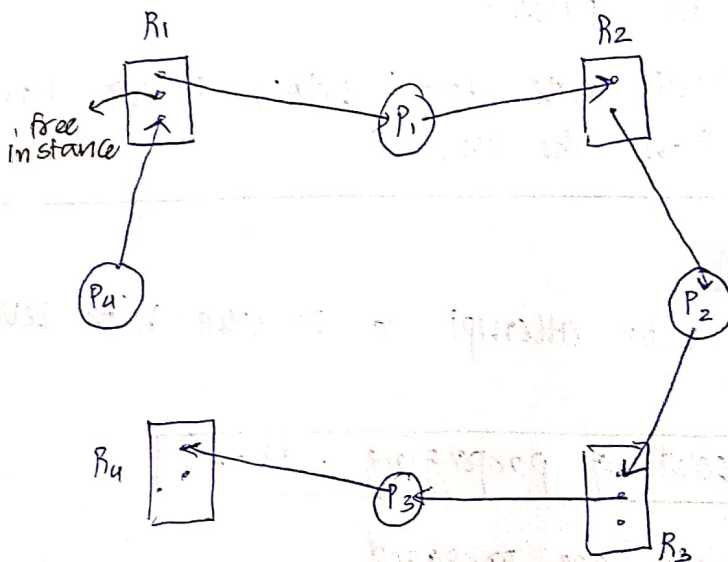
$s \rightarrow$ pages are prepaged
 α fraction of s pages
 $0 \leq \alpha \leq 1$

- * Page size
- * Inverted Page Table
- * Programme structure : compatible with the compiler
- * I/O Interlock : Some of the pages are locked / fixed in M/M & used by all the processes.

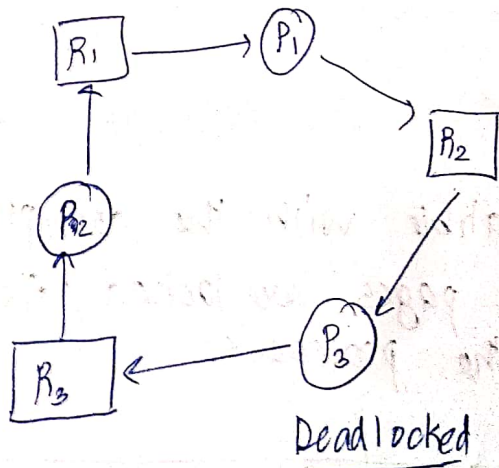
→ DEADLOCK



Request
Use
Release



Resource Allocation Diagram



Deadlock Conditions

- (i). Mutual exclusion
- (ii). Hold and Wait
- (iii). No preemption
- (iv). Circular wait

$V \rightarrow$ set of vertices $\{V_1, V_2, \dots, V_n\}$

$P \rightarrow$ set of processes $\{P_1, P_2, \dots, P_n\}$

$R \rightarrow$ set of resources $\{R_1, R_2, \dots, R_n\}$

Two types of edges: $P_i \rightarrow R_j$ (process P_i is requesting resource R_j)
Assignment Edges $R_j \rightarrow P_i$ (process P_i is holding resource R_j)

$P_i \rightarrow$ nodes

$R_i \rightarrow$ 

instances of Resources

→ Methods for handling Deadlock

- * either deadlock has never occurred
- * deadlock entered → prevent it
or
avoid it

→ Deadlock prevention

(i). Mutual exclusion
(for non-sharable resources)

(ii). Hold & Wait

~~whenever~~ (A process requests a resource only when it is available & not held by any other process)

- Protocols

→ PROBLEMS

(i). unused resource

(ii). starvation

(iii). No preemption

No another process will start its execution until the previous process has ~~not~~ completed its execution.

(iv). Circular Wait

Ordering of processes is fixed.

→ Deadlock Avoidance

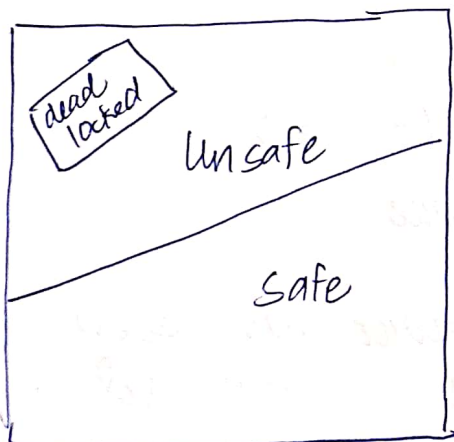
Check for every request if the system goes into deadlock. If the system goes into deadlock, do not allocate the resource.

→ resource allocation graph

Safe state: if the system can allocate the resources to processes and can still avoid deadlock.

Safe sequence

$\langle P_1, P_2, \dots, P_n \rangle$ if for each process P_i , the process P_i can still request a resource which can be satisfied by the remaining resources



* In unsafe state, OS/ process cannot request a resource otherwise the system goes into deadlock state.

Example: 10 magnetic disks
3 processes

Process	Disk Req'd.	Disks Allocated at time T_0
P_0	10	5
P_1	4	2
P_2	9	2
		<u>9</u>

$10 - 9 = 1$ disk is available

At time t_1 , $\left. \begin{array}{l} P_0 = 2 \\ P_1 = 1 \\ P_2 = 3 \end{array} \right\}$ disks requested.

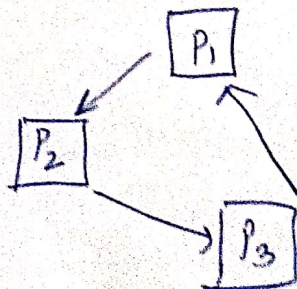
At time t_1 , only P_1 will be allocated
 P_1 is in safe state.

* Resource Allocation Graph

Claim edge: dashed line ($--->$)

$\rightarrow$ claiming the resource

Request edge: When resource has been allocated
($\rightarrow$)
a claim edge is converted into request resource edge.



→ BANKERS ALGO

↳ Multiple instances of each resource

Available
Max
Allocation
Need

Safety algo

1. Work & finish
Initialize $work = Available$
 $finish[i] = false$ for $i = 1 \dots n$

2. Find an i s.t.

(a). $finish[i] = false$

(b). $Need[i] \leq work$

if no such i exists go to step 4.

3. $Work = Work + Allocation$

$finish[i] = true$

Go to step 2.

4. if $finish[i] = true$ for all i ,
then system is in safe state.

* Resource Allocation Request Algo

1. If $Request\ i \leq Need\ i$
Go to step 2.

Otherwise

raise an error.

2. If $Request\ i \leq Available$
go to step 3.

3. Have the system allocated the request.

$Available = Available - Request\ i$

$Allocation\ i = Allocation\ i + Request\ i$

$Need\ i = Need\ i - Request\ i$;

Eg: 5 processes (P_1, P_2, P_3, P_4, P_5)
 3 Resources (A, B, C)
 Resource A $\rightarrow$ 10 instances
 B $\rightarrow$ 5
 C $\rightarrow$ 7

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_1	0	1	0	7	5	2	3	2	2
P_2	2	0	0	3	2	2	$\rightarrow 10 - (\sum \text{Allocation}_A)$ $10 - (2+2+2) = 3$		
P_3	3	0	2	9	0	2			
P_4	2	1	1	2	2	2			
P_5	0	0	2	4	3	3			

	Need		
	A	B	C
P_1	7	4	3
P_2	1	2	2
P_3	6	0	0
P_4	0	1	1
P_5	4	3	1

The system is in safe state.
 Then a request is made
 by process P_1 .

(P_1). Request $\langle 1 \ 0 \ 2 \rangle$

$\rightarrow \leq$ Available

Available = 2, 3, 0

(P_2). Request $\langle 2 \ 0 \ 2 \rangle$

$\rightarrow >$ Available

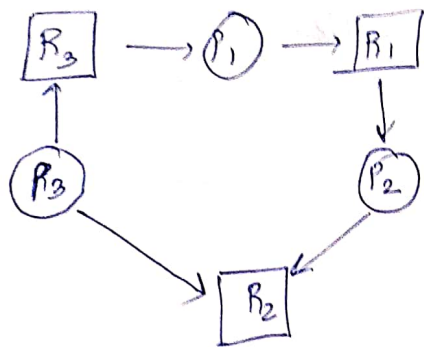
$\therefore$ system is not in safe state.
 if this request is allocated.

→ DEADLOCK DETECTION

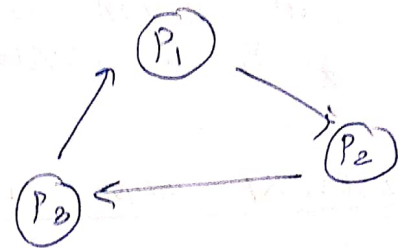
↳ SAFE STATE: no deadlock has been detected
UNSAFE STATE: deadlock has been detected

* SINGLE INSTANCE OF EACH RESOURCE

Wait-for graph



Resource Allocation Graph



Wait-for-graph (deadlock)

since a cycle is being formed, it is in deadlock

* SEVERAL INSTANCES

Available, Allocation, Request

1. Initialize work & finish.

work: Available for all $i = 1 \dots n$.

then $Finish[i] = \text{false}$
otherwise

$Finish[i] = \text{true}$

2. Find an i s.t.

(a). $Finish[i] = \text{false}$

(b). $Request[i] \leq work$

If no such i exists, go to step 4.

3. $work = work + Allocation;$
 $finish[i] = true;$
Go to step 2.

4. If $finish[i] = false$
for some $i, 1 \leq i \leq n$,
then system is in deadlock state.
P_i is deadlocked.

→ RECOVERY FROM DEADLOCK

- (i). abort process
- (ii). Preempt the resource

Termination

- (a). All processes are aborted
- (b). Abort one process at a time.
(PARTIAL TERMINATION)

Preemption

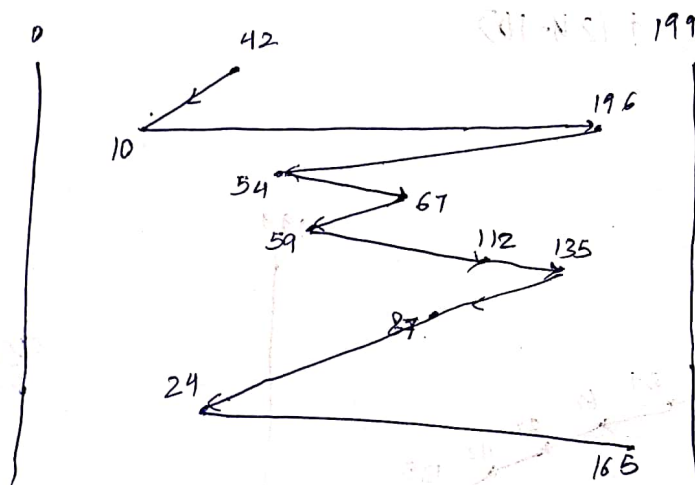
- (a). Select a victim.
- (b). Roll back
- (c). No starvation

→ DISK SCHEDULING / MANAGEMENT

1. FCFS
2. SSTF
3. SCAN (Elevator's Algorithm)
4. C-SCAN
5. LOOK
6. C-LOOK

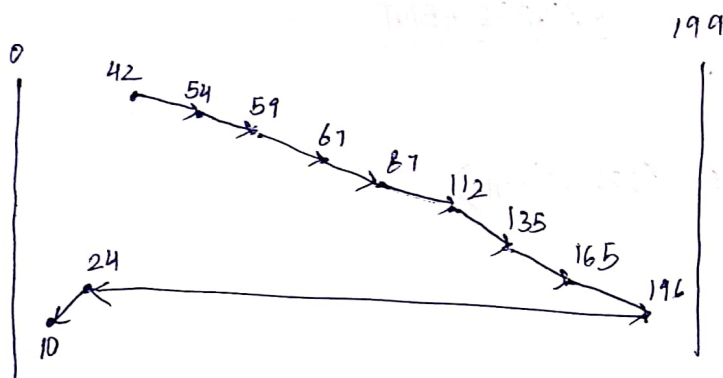
head location → 42

Move → 10, 196, 54, 67, 59, 112, 135, 87, 24, 165



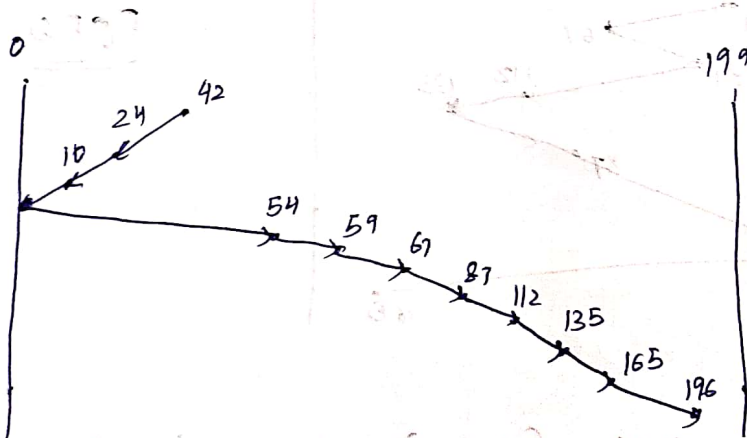
FCFS.

$$\begin{aligned}
 \text{Total head movements} &= (42-10) + (196-10) + (196-54) \\
 &\quad + (67-54) + (67-59) + (112-59) + \\
 &\quad + (135-112) + (135-87) + (87-24) + \\
 &\quad + (165-24)
 \end{aligned}$$



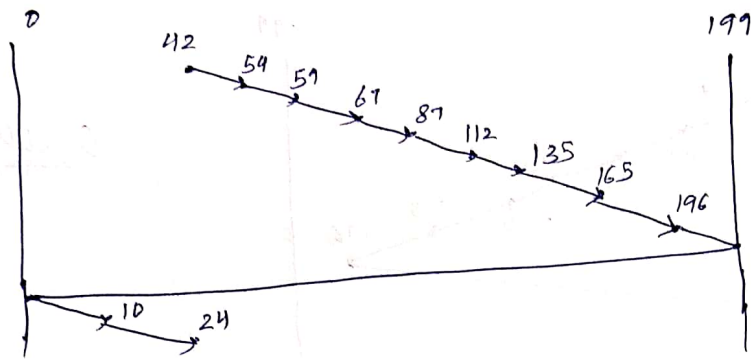
SSTF

$$\begin{aligned} \text{Total head movements} &= (54-42) + (59-54) + (67-59) + (87-67) + (112-87) \\ &\quad + (135-112) + (165-135) + (196-165) + (196-24) \\ &\quad + (24-10) \end{aligned}$$



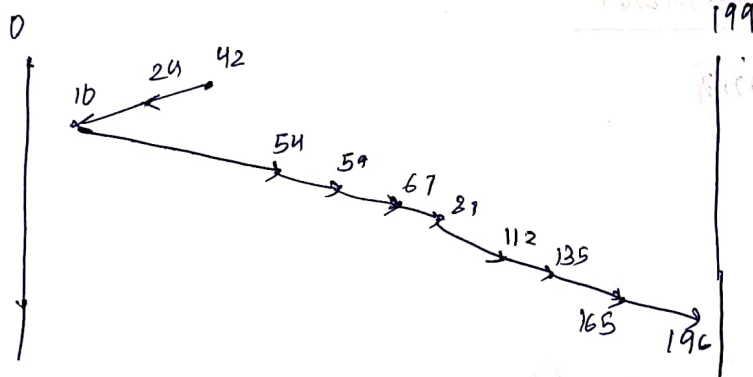
SCAN

$$\begin{aligned} \text{Total head movements} &= (42-24) + (24-10) + (10-0) + (54-0) \\ &\quad + (59-54) + (67-59) + (87-67) + (112-87) \\ &\quad + (135-112) + (165-135) + (196-165) \end{aligned}$$



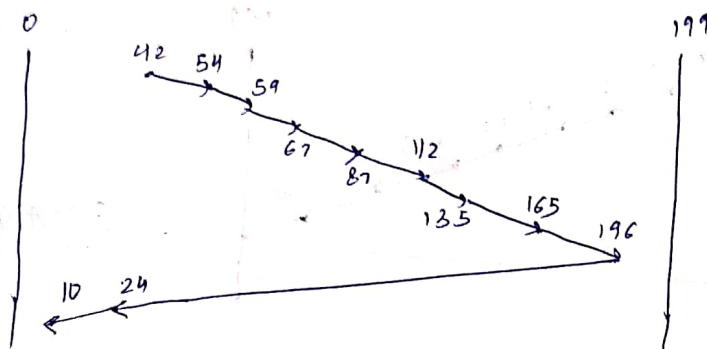
GSCAN

$$\begin{aligned} \text{Total head movements} &= (54-42) + (59-54) + (67-59) + (87-67) \\ &+ (112-87) + (135-112) + (165-135) + (196-165) \\ &+ (199-196) + (199-0) + (10-0) + (24-10) \end{aligned}$$



LOOK

$$\begin{aligned} \text{Total head movements} &= (42-24) + (24-10) + (54-10) + (59-54) \\ &+ (67-59) + (87-67) + (112-87) + (135-112) \\ &+ (165-135) + (196-165) \end{aligned}$$



C-LOOK

$$\begin{aligned} \text{Total head movements} = & (54 - 42) + (59 - 54) + (67 - 59) + (87 - 67) \\ & + (112 - 87) + (135 - 112) + (165 - 135) + \\ & (196 - 165) + (196 - 24) + (24 - 10) \end{aligned}$$

→ DISK MANAGEMENT

* Disk formatting

* Boot Block

MS-DOS

* Bad Blocks

└ Avoid

└ Format - Replacing

* Sector sparing

* Sector slipping

* Swap space management

└ swap space use

└ swap space location

* RAID structure (Redundant Arrays of Inexpensive Disks)

- * mirroring / shadowing
- * Parallelism - bit-level - striping
- * Block-level striping

* RAID Levels

- Level 0 : non-redundant striping
 - Level 1 : mirrored disks
 - Level 2 : memory style error correction
 - Level 3 : bit interleaved parity
 - Level 4 : block interleaved parity
 - Level 5 : block interleaved distributed parity
 - Level 6 : P+Q redundancy
-